map-charting-student-math-misunderstandings

10位 Solution

Rank	10
Team	Liwen Zheng
Author	Levin
Topic ID	612038
Version	Japanese Translation

Source: https://www.kaggle.com/competitions/map-charting-student-math-misunderstandings/discussion/612038
Retrieved with Kaggle CLI on 2026-10-21.

Kaggle Discussionの主投稿と、技術的補足を含む選定済みQ&Aを日本語へ翻訳した版です。code、URL、model名、識別子、数値は原文を保持しています。

主催者とKaggleに大きな感謝を。メダルを目指した初めての参加で、初めてのメダル、しかもソロ金メダルです。以下に手法を簡潔にまとめます。

概要

タスクを直接の多クラス分類として扱い、QLoRAでfine-tuningした15個のモデル（5-fold × 3つのbackbone: Qwen3-Reranker-8B、Qwen3-Embedding-8B、Qwen2.5-32B-Instruct）をアンサンブルしました。

主な要素

- データの衛生管理とfoldの制御: True/Falseの不整合を自動修正し、近似重複を同じfoldに入れる。
- long-tailのクラスを安定させるため、class weightのwarmup（約33%）付きのFocal + CE混合loss。
- 推論の高速化: Qwen2.5-32B-Instructについて、LoRAをmerge → base/cls-headを分割 → GPTQ-4bitのbase + FP16のclassifier head → vLLM embed → logits → アンサンブル。

モデルと学習

Backbone（すべて `AutoModelForSequenceClassification` を使用）

1. Qwen3-Reranker-8B (QLoRA)
2. Qwen3-Embedding-8B (QLoRA)
3. Qwen2.5-32B-Instruct (QLoRA)

各backboneを5 foldで学習 ⇒ 合計15モデル。

Lossと最適化

- Loss: Focal（難しい例を重視するため、smoothingなしのCEで計算）と、calibrationのためのlabel smoothing = 0.1付きCross-Entropyの混合。クラスごとの重みはwarmup後に適用。
- Warmup: long tailでの初期の不安定な更新を避けるため、全ステップの `weight_warmup_ratio ≈ 0.33`。
- 量子化を考慮した学習: 8Bモデルはbitsandbytesの4-bit NF4 + gradient checkpointingを使用（QLoRA方式）。

ハイパーパラメータ

- Qwen3-Reranker-8B / Qwen3-Embedding-8B（単一のRTX 4090） `lr=1e-4`, `epochs=4`, scheduler=`cosine`, `weight_warmup_ratio=0.33`, 実効global batch size=16。
- Qwen2.5-32B-Instruct（A800-40GB） `lr=1e-4`, `epochs=3`, `weight_warmup_ratio=0.33`, 実効global batch size=32。

LoRAの設定（典型例）：`r=64, alpha=128, dropout=0.05`、Q/K/V/OとMLPのprojectionを対象とし、classifier head（`score`）を保存。

推論のエンジニアリング

8Bモデル

- LoRAはmergeしません。4-bitに量子化したbaseを保存し、推論時にLoRA adapterと一緒に読み込むことで、各モデルを単一のGPUで効率的に扱えるようにしました（T4 2枚でのスケジューリングとも相性が良い）。

32Bモデル

- ワークフロー: LoRAをmerge → baseとclassifier headを分割 → baseをGPTQ-4bitに量子化（cls-headはFP16のまま）→ vLLMを `embed` モードで実行 → 外部のLinear headを適用してlogitsを生成 → logitsを書き出す。

```
():

  ()
  ()
    os.makedirs(args.base_model_save_path, exist_ok=)
    os.makedirs(args.classifier_head_save_path, exist_ok=)

    full_model = AutoModelForSequenceClassification.from_pretrained(
        args.merged_model_path,
        device_map=,
        torch_dtype=torch.float16
  )
    tokenizer = AutoTokenizer.from_pretrained(args.merged_model_path)

    base_model = full_model.model
    classifier_head = full_model.score

  ()
    base_model.save_pretrained(args.base_model_save_path)
    tokenizer.save_pretrained(args.base_model_save_path)

  ()
    torch.save(classifier_head.state_dict(), os.path.join(args.classifier_head_save_path, ))

    classifier_config = {
        : full_model.config.hidden_size,
        : full_model.config.num_labels,
        : full_model.config.id2label,
        : full_model.config.label2id,
        : classifier_head.bias    ,
  }
     (os.path.join(args.classifier_head_save_path, ), )  f:
        json.dump(classifier_config, f)

     shutil
    shutil.rmtree(args.merged_model_path)
  ()

  ()

():
     vllm  LLM
```

```
  ()
  ()
  ()

     (os.path.join(args.classifier_head_path, ), )  f:
        classifier_config = json.load(f)

    classifier_head = nn.Linear(
        classifier_config[],
        classifier_config[],
        bias=(classifier_config.get(, ))
  )
    state = torch.load(os.path.join(args.classifier_head_path, ), map_location=)
    classifier_head.load_state_dict(state)
    classifier_head.to().()

    llm = LLM(
        model=args.quantized_model_path,
        quantization=,
        dtype=,
        task=,
        override_pooler_config={: , : },
        tensor_parallel_size=
  )
  ()

    le = LabelEncoder()
    train_df = pd.read_csv(args.label_data_path)
    train_df.Misconception = train_df.Misconception.fillna()
    train_df[] = train_df.Category +  + train_df.Misconception
    le.fit(train_df[])

    all_results = []
  :
        embed_outputs = llm.embed(TEST_PROMPTS)
        embeddings = torch.from_numpy(np.stack([np.array(o.outputs.embedding, dtype=np.float32)  o
embed_outputs])).to()

         torch.no_grad():
            logits = classifier_head(embeddings)
            probs = torch.softmax(logits, dim=-).cpu().numpy()
            top_indices = np.argsort(-probs, axis=)

         i  ((TEST_PROMPTS)):
            pred_id = top_indices[i, ]
            top_3_ids = top_indices[i, :]
            result = {
                : i,
                : TEST_PROMPTS[i],
                : le.inverse_transform([pred_id])[],
                : (probs[i, pred_id]),
                : le.inverse_transform(top_3_ids).tolist(),
                : probs[i, top_3_ids].tolist()
      }
            all_results.append(result)
      ()

  :
        llm, classifier_head; gc.collect(); torch.cuda.empty_cache()

     (args.output_json_path, )  f:
        json.dump(all_results, f, indent=)
  ()
```

```
()
```

- 量子化: `group_size=64`、symmetric、GAR（Group-Aware Reordering）、`true_sequential=True` のGPTQ-4bit。学習用プロンプトと同じ形式の約2kのcalibrationテキスト。
- vLLMの設定: `quantization="gptq"`, `dtype="half"`, `task="embed"`, `override_pooler_config={"pooling_type": "LAST", "normalize": False}`。

アンサンブル

logitの加重ブレンド（32Bの各foldは重み2、8Bの各foldは重み1）を使い、MAP@3のためにTop-3を取りました。これは確率の平均より安定していました。

（私の場合）うまくいかなかったこと

- ハードな論理制約付きの階層的multi-task分類器
- 32Bモデルをpointwise rerankerとして使う
- 32Bモデルをlistwise rerankerとして使う
- TTA

ラベルノイズ、long tail、MAP@3という目的を考えると、きれいなsplit + バランスの取れた学習 + 多数の控えめなモデルが、より重いパイプラインに勝りました。

結果

Setting	Public MAP@3	Private MAP@3
Single 8B model	0.946	0.942
8B backbones × 5 folds	0.950	0.946
32B-Instruct × 5 folds	0.950	0.947
Ensemble (15 models)	0.951	**0.948**

謝辞と参考

データの衛生管理、学習、量子化、アンサンブルの選択に役立ったdiscussion、notebook、write-upを共有してくださったKaggleコミュニティに深く感謝します。特に:

- brendanartley — discussion
- kishanvavdara — notebook: Ensemble Gemma/Qwen/DeepSeek

- cdeotte — notebook: Gemma2-9B-IT CV 0.945
- jaytonde — notebook: DeepSeekMath-7B LB 0.944
- ebinan92 — 5th-place solution write-up
- yannan90 — 8th-place solution write-up
- sayoulala — discussion
- nikitababich — 6th-place solution write-up
- conjuring92 — 1st-place detailed solution)

補足Q&A

common-people — 質問

共有ありがとうございます。 いくつか質問があります。 これら3つのモデルはどのように動作するのでしょうか。また、rerankerモデルの学習データはどうなっていますか（データの偏りなどの問題をバランスさせるために何か工夫が必要か）。pairwiseとlistwiseのどちらでしょうか。説明をお願いします。ありがとうございます。 Qwen3-Reranker-8B (QLoRA) Qwen3-Embedding-8B (QLoRA) Qwen2.5-32B-Instruct (QLoRA)

Levin — 回答

3つのモデルはすべて、cdeotte さんの notebook: Gemma2-9B-IT CV 0.945 と同じように、AutoModelForSequenceClassificationで読み込んだ分類モデルです。

pairwiseとlistwiseのrerankerについては、ebinan92 さんの 5th-place solution write-up を参考にしました。

具体的には、listwise rerankerでは、まずrecallモデルが生成した各サンプルの予測結果から上位5候補を取りました。次に、いくつかの情報（上位5候補の説明や、その問題のすべての選択肢など）をプロンプトに連結しました。最後に、この結合した入力をAutoModelForCausalLMに入力し、選択肢A、B、C、D、Eを出力させました。

しかし、この手法では理想的な結果は得られませんでした。私の場合、14Bの5-foldモデルでの最良の性能はPublicデータセットで0.948、Privateデータセットで0.945でした。私の方法が間違っていたのかもしれませんが、いずれにせよ最終的にrerankerの手法は諦めました。

Bilzard — 質問

@zhengliwen ソロ金メダルおめでとうございます。

珍しいクラスの扱い方に興味があります。

> Loss: Focal（難しい例を重視するため、smoothingなしのCEで計算）と、calibrationのためのlabel smoothing = 0.1付きCross-Entropyの混合。クラスごとの重みはwarmup後に適用。 Warmup: long tail での初期の不安定な更新を避けるため、全ステップの weight_warmup_ratio ≈ 0.33。

1. `warmup_ratio` に加えて、独立したwarmupパラメータ `weight_warmup_ratio` を使っているのですか？
2. クラスごとの重みとは、各クラスごとの(alpha, gamma)という意味ですか？これらのパラメータはどのように選んだのでしょうか、手動ですか？そうであれば、パラメータを共有していただけますか？

$$ \mathrm{FL}(p_t) = - \alpha_t (1 - p_t)^{\gamma} \log(p_t) $$

Levin — 回答

メインのwarmupに加えて、別個のweight warmup（全ステップの約33%）を使っています。

```
():
  ():
        ().__init__(*args, **kwargs)
        .combined_loss_fn = combined_loss_fn
        .accelerator = accelerator

  ():
        labels = inputs.get()
         .combined_loss_fn     (.combined_loss_fn, ):

       :
                current_step = .state.global_step
            Exception:
                current_step =
              .combined_loss_fn.set_step(current_step)
        labels = inputs.pop(, )
        outputs = model(**inputs)
        logits = outputs.logits

         .combined_loss_fn   :
            loss_dict = .combined_loss_fn(logits, labels)
             (loss_dict, ):
                loss = loss_dict[]
                 .accelerator    .accelerator.is_main_process:
                    wandb.log({
                        : loss_dict[].item(),
                        : loss_dict[].item(),
                        : loss.item()
           })
       :
                loss = loss_dict
    :
            loss = outputs.loss

         (loss, outputs)  return_outputs  loss

(nn.Module):
  ():
        ().__init__()
        .focal_loss = FocalLoss(
            alpha=alpha,
            gamma=gamma,
    )
        .ce_loss_fn = nn.CrossEntropyLoss(label_smoothing=label_smoothing, reduction=)
        .focal_weight = focal_weight
        .ce_weight = ce_weight
        .class_weights = class_weights
        .weight_warmup_steps = weight_warmup_steps
        .current_step =

  ():
        .current_step = step
        .focal_loss.set_step(step)

  ():
         .class_weights    .current_step < .weight_warmup_steps:

         .class_weights
```

```
    ():
        effective_weights = .get_effective_weights()
         effective_weights   :
            effective_weights = effective_weights.to(inputs.device)

        focal = .focal_loss(inputs, targets)

        ce_loss = .ce_loss_fn(inputs, targets)
         effective_weights   :
            class_weight_per_sample = effective_weights[targets]
            ce_loss = ce_loss * class_weight_per_sample
        ce = ce_loss.mean()

        combined = .focal_weight * focal + .ce_weight * ce

     {
            : combined,
            : focal.detach(),
            : ce.detach(),
            : effective_weights
    }

 (nn.Module):
    ():
        ().__init__()
        .alpha = alpha
        .gamma = gamma

    ():
        ce = F.cross_entropy(inputs, targets, reduction=)
        pt = torch.exp(-ce)
        focal_factor = ( - pt) ** .gamma

        focal_loss = .alpha * focal_factor * ce
        focal_loss = focal_loss.mean()

         focal_loss

combined_loss_fn = CombinedLoss(
        focal_weight=args.focal_weight,
        ce_weight=-args.focal_weight,
        alpha=,
        gamma=,
        label_smoothing=args.label_smoothing,
        num_classes=n_classes,
        class_weights=class_weights_tensor,
        weight_warmup_steps=WEIGHT_WARMUP_STEPS,
  )
```

common-people — 質問

なるほど、ありがとうございます。では、8Bのrerankerはpairwiseなのですね。k-foldのrecallモデルを使って、その上位2候補の学習データを生成しているのでしょうか？私もlistwise rerankerの手法を試しましたが、私のコードではうまくいきませんでした。

!!!!! — 質問

おめでとうございます！32Bモデルの学習にどれくらい時間がかかるか伺ってもよいですか？実効global batch sizeが32ということは、gradient accumulationを使ったのでしょうか？

Levin — 回答

ありがとうございます！私の学習（3 epoch、実効global batch = **32**）では: **2× L20 40GB** GPUあたりのbatch = **16**、**gradient accumulationなし**（GA=1）→ **約5時間**。 **2× A100 40GB** 同じ設定（GPUあたりのbatch 16、GA=1）→ **約3時間**。 **4× A100 40GB** GPUあたりのbatch = **8**、**gradient accumulationなし**（8×4=32、GA=1）→ **約2時間**。

Levin — 回答

k-foldのrecallモデルを使って上位5候補を作り、それでlistwise rerankerを学習しています。それでも強い分類器を上回りませんでした。 うまくいかない場合は、(1) fold間のリーク、(2) next-tokenの位置合わせ（正しいラベルのトークンを選んでいるか）を再確認してください。

Gaurav Rawat — 質問

また、hybridな損失関数への変更はどのように判断していますか？うまくいくこともありますが、損失関数を変えるときに何か根拠があるのでしょうか、それともCVで効くものを試行錯誤するだけですか？

Levin — 回答

ありがとうございます！ほぼCV優先です。CE（+ smoothing）から始めて、少しずつ調整し、改善したものを残します。

Roschild.Rui — 質問

@zhengliwen おめでとうございます！ vllmのどのバージョンを使っていますか？T4で14Bの量子化モデルをembモードで推論できるように、ソースコードを何か変更しましたか？

Levin — 回答

ソースの変更は不要です。pip install vllmするだけです。標準のpipリリース（ローカルのパッチなし）を使っています。vLLMのembedding APIは、サポートされているdecoder-onlyモデルならそのまま動作します。

サポートされているモデルの一覧はここで確認できます（LLM.embed）: https://docs.vllm.ai/en/latest/models/supported_models.html#embedding

Roschild.Rui — 質問

ありがとうございます！具体的にはvllmのどのバージョンですか？vllm 0.11.0で `task=emb` を設定し、2 GPUで並列推論すると問題が起きました。

Levin — 回答

ありがとうございます！私もnotebookでvLLM 0.11.0（pip）を使っていて、task="embed"は動作しています。

2×GPUの場合、次の点を再確認してください。

- tensor_parallel_size=2を設定する。

- backboneがGPTQ-4bitなら、quantization="gptq"とpooler（例: override_pooler_config={"pooling_type":"LAST","normalize":False}）を渡す。
- モデルがLLM.embedのサポート一覧に含まれていることを確認する。